

Kafka & Event Streaming

Complete guide to Kafka topics, producers, consumers, and event flows

1. What is Apache Kafka?

Apache Kafka is a distributed event streaming platform. In K Star, Kafka acts as the backbone for asynchronous communication between microservices. When a transfer is made, an event is published to Kafka — other services (notification, audit) can independently consume this event without the transfer service needing to know about them.

■ Core Kafka Concepts

Concept	Definition	K Star Example
Topic	Named channel where events are published	transfer-events, nacha-payments, user-events, nft-trades
Producer	Service that publishes events to a topic	TransferService publishes to transfer-events
Consumer	Service that reads events from a topic	NotificationConsumer reads from transfer-events
Partition	Parallel lanes within a topic for scaling	Each topic has 1 partition (demo; use 3+ in prod)
Offset	Position of a message in a partition	Tracked per consumer group for replay capability
Consumer Group	Set of consumers sharing a topic load	notification-consumer-group for transfer-events
Broker	Kafka server node that stores messages	Running as Docker container on port 9092
Zookeeper	Coordinates the Kafka cluster	Required for Kafka 7.x (KRaft removes this in Kafka 3+)

2. Kafka Topics in K Star

Topic Name	Producer	Consumer	Event Type	Purpose
user-events	user-service	(future: audit-service)	USER_REGISTERED	Notify on new user registration
transfer-events	transfer-service	NotificationConsumer	TRANSFER_INITIATED, COMPLETED	Notify P2P transfer
nacha-payments	nacha-service	(future: settlement-service)	INITIATED, SUBMITTED, REJECTED	Track payment lifecycle
nft-trades	nft-service	(future: royalty-service)	MINTED, LISTED, TRADED, BURNED	Track ownership changes
rate-updates	exchange-service	(future: alert-service)	RATE_UPDATED	Broadcast exchange rate changes

3. Transfer Event Flow — Step by Step

The UPI P2P transfer is the most complex event flow in K Star. Here is every step from user action to database update:

- Step 1: User submits POST /api/transfer/upi with fromUpId, toUpId, amount, pin
- Step 2: Gateway validates JWT → adds X-User-Name header → forwards to transfer-service:8083
- Step 3: TransferService.initiateTransfer() validates: UPI IDs exist, PIN is correct, balance sufficient
- Step 4: Transaction entity created with status=PENDING, saved to MySQL
- Step 5: TransferEvent published to Kafka topic "transfer-events" via KafkaTemplate
- Step 6: NotificationConsumer @KafkaListener consumes event from "transfer-events" topic
- Step 7: Consumer deducts amount from sender wallet, credits receiver wallet
- Step 8: Transaction status updated to COMPLETED (or FAILED on error)
- Step 9: Response returned to user with transaction ID and status

■ TransferEvent Schema

```
public class TransferEvent {  
    private String transactionId; // UUID  
    private String fromUpId; // e.g. "9876543210@kstar"  
    private String toUpId; // e.g. "1234567890@kstar"  
    private BigDecimal amount; // e.g. 500.00  
    private String currency; // "INR"  
    private String status; // "INITIATED", "COMPLETED", "FAILED"  
    private LocalDateTime timestamp; // Event creation time  
}
```

4. Kafka Configuration

■ Producer Config (all services)

```
spring:  
kafka:  
bootstrap-servers: kafka:9092 # Docker service name  
producer:  
key-serializer: org.apache.kafka.common.serialization.StringSerializer  
value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
```

■ Consumer Config (transfer-service)

```
spring:  
kafka:  
consumer:  
group-id: transfer-service  
key-deserializer: org.apache.kafka.common.serialization.StringDeserializer  
value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer  
properties:
```

```
spring.json.trusted.packages: ""
```

■ Consumer Annotation

```
@KafkaListener(topics = "transfer-events", groupId = "transfer-service")
public void onTransferEvent(TransferEvent event) {
    log.info("Processing transfer: {} -> {} : ■{}",
        event.getFromUpiId(), event.getToUpiId(), event.getAmount());
    // Process debit/credit logic here
}
```

5. Why Kafka Over Direct REST Calls?

Aspect	Direct REST	Kafka Event
Coupling	Tight — sender must know receiver URL	Loose — services unaware of each other
Failure	If receiver is down, sender fails	If receiver is down, event waits in topic
Scalability	Manual load balancing	Kafka partitions auto-distribute load
Audit	Manual logging required	Every event is persisted in topic (configurable retention)
Replay	Not possible	Can replay from any offset for debugging
Speed	Synchronous — waits for response	Asynchronous — fire and forget